

Sistemas Distribuídos: Atividades 01

1 Descrição

Leia antes de ver os atividades:

- **Data/Hora da Entrega:** 25/11/2025, sexta-feira, até às 19h (FIRME).
- **Entregáveis:** Enviar o link do Github com o código organizado e o README contendo os passos de como rodar o código (Exemplo: <https://github.com/davidjmc/midd4vc-isc225>)
- Apenas um dos membros do grupo precisa realizar a entrega.
- A equipe pode usar qualquer linguagem de programação.
- As ordens das apresentações serão impreterivelmente por sorteio, mas todos os grupos devem me entregar até a hora definida acima.
- Serão realizadas apenas **4 apresentações de 20 minutos**, sorteadas entre os 12 grupos.
Rotação de Apresentações: Os grupos que apresentarem nesta primeira atividade não serão sorteados para a apresentação na segunda atividade nem na terceira, garantindo que grupos diferentes apresentem em cada ciclo de atividades.
- Não precisa de slide; a apresentação deve focar na explicação do código e na execução prática. Lembrem-se: funcionou ou não funcionou é o que importa.
- **IMPORTANTE:** Não haverá dia de acompanhamento da atividade. Portanto, se você tiver com alguma dúvida sobre como realizar a implementação, faça uma escolha a partir do que você está entendendo do projeto. O importante é enviar a implementação no prazo estabelecido e da forma como você entendeu.

2 Implementações por Grupos

[Equipe 01] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que funcione como um repositório de imagens colaborativo. O servidor deve permitir o upload, armazenamento, listagem, visualização e download de imagens entre vários usuários. Cada cliente pode enviar suas

fotos ou figuras e acessar as imagens compartilhadas por outros. O cliente se conecta ao servidor e, opcionalmente, fornece um nome de usuário/senha para identificar e registrar a autoria das imagens. O servidor mantém um histórico dos arquivos enviados. O cliente envia comandos (ex.: `UPLOAD|nome_arquivo.jpg|tamanho_em_bytes`). Em seguida, ele envia os bytes da imagem, aproveitando a confiabilidade do TCP para transferência. O servidor cria um diretório organizado por usuário ou por data, grava a imagem e confirma a operação. Com o comando `LIST`, o cliente obtém uma relação das imagens disponíveis, incluindo autor, data de envio, tamanho e miniatura (thumbnail) opcional. Ao receber o comando `DOWNLOAD|nome_arquivo.jpg`, o servidor envia os bytes do arquivo. Por fim, um comando `VIEW|nome_arquivo.jpg` pode retornar apenas uma versão reduzida (thumbnail) para pré-visualização.

[Equipe 02] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que gerencie uma agenda de compromissos compartilhada em memória. O servidor deve garantir conexões persistentes e permite que vários usuários acessem e manipulem a agenda compartilhada em tempo real. O servidor deve manter em memória uma lista de eventos contendo informações como título, data, hora, duração e descrição opcional. Cada cliente conectado poderá inserir, listar, atualizar ou remover compromissos através de comandos bem definidos, como `ADD <data> <hora> <título>`, `LIST <data>`, `UPDATE <id> <novo_valor>` ou `DELETE <id>`. É essencial que o servidor se preocupe com a integridade dos dados da agenda, implementando mecanismos de concorrência para evitar falhas em operações simultâneas.

[Equipe 03] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que faça transmissão contínua (streaming) de áudio, como MP3 ou WAV, utilizando socket TCP para garantir entrega confiável e ordenada dos dados. O servidor deve carregar um arquivo de áudio escolhido e dividi-lo em blocos (por exemplo, 8 KB ou 16 KB), enviando-os de forma sequencial e constante para cada cliente conectado. O cliente, por sua vez, deve ser capaz de reproduzir o áudio enquanto os blocos chegam, sem esperar que o arquivo inteiro seja transferido. Essa reprodução em tempo real exige a implementação de um buffer de reprodução no cliente, que armazena temporariamente alguns segundos de áudio para compensar eventuais variações de rede (jitter).

[Equipe 04] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que funcione como um feed de notícias contínuo (modelo PUB/SUB), onde os clientes recebem atualizações assim que novas notícias são publicadas. O servidor mantém uma lista de categorias (por exemplo: tecnologia, esportes, cultura) e um repositório em memória ou em arquivo para armazenar as notícias recebidas de um “editor” ou de um processo automatizado. Assim que uma nova notícia é cadastrada, o servidor deve imediatamente transmitir o título, o resumo e a categoria para todos os clientes que se inscreveram naquele tema, sem que eles precisem fazer uma requisição extra. Os clientes se conectam ao servidor, escolhem as categorias de interesse (ex.: `INSCREVER tecnologia`, `esportes`) e ficam ouvindo as atualizações em um socket TCP persistente. Eles devem ser capazes de alterar as assinaturas durante a sessão (ex.: `REMOVER tecnologia` ou `INSCREVER cultura`). O

servidor precisa gerenciar várias conexões simultâneas, armazenar as preferências de cada cliente e enviar apenas as notícias relevantes para cada um.

[Equipe 05] Implemente um sistema distribuído de “conversão de arquivos”, usando cliente-servidor e Sockets TCP. O servidor permite a conversão de arquivos de um formato para outro de maneira centralizada. Ele recebe arquivos enviados pelos clientes, processa a conversão conforme o formato solicitado e retorna o novo arquivo. Exemplos simples de conversão incluem (escolha apenas uma delas): de .txt para .pdf ou de .png para .jpg; de .wav para .mp3 (áudio). O cliente envia uma requisição inicial como: CONVERT <formato_origem> <formato_destino> <nome_arquivo>. O servidor confirma se a conversão é suportada antes de receber o arquivo e invoca uma biblioteca ou rotina interna para realizar a transformação. O TCP é fundamental para garantir que o arquivo de entrada seja transferido integralmente e sem erros antes de ser processado, e que o arquivo convertido retorne ao cliente com a mesma garantia de entrega. O cliente salva em disco e exibe mensagem de sucesso ou falha.

[Equipe 06] Implemente um sistema cliente-servidor usando socket TCP que atua como um agente de monitoramento de desempenho, coletando e exibindo, em tempo quase real, métricas de vários computadores. Cada cliente é instalado em uma máquina a ser monitorada e atua como um agente, reunindo periodicamente dados como (escolha apenas dois para monitorar): uso de CPU (percentual de ocupação por núcleo e média geral), memória (total, utilizada, livre), disco (uso de espaço, taxa de leitura/escrita), ou rede (taxa de upload/download, pacote/s perdidos). Essas informações são enviadas em intervalos configuráveis (ex.: a cada 5 segundos) para o servidor, que mantém conexões persistentes com todos os agentes. O servidor, por sua vez, mantém uma lista de clientes conectados e armazena os dados em memória para visualização.

[Equipe 07] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que gerencie uma votação, aceitando múltiplos clientes participarem de uma votação em tempo real. O servidor gerencia a votação, mantém o registro de votos de cada usuário e envia atualizações contínuas sobre o resultado parcial a todos os clientes conectados. Cada cliente se conecta ao servidor e informa um identificador único (nome ou ID); o servidor verifica se o usuário já votou, garantindo que cada pessoa possa votar apenas uma vez. Os clientes enviam seu voto em um formato padronizado, por exemplo: VOTE opcao_escolhida. O servidor valida o voto e confirma o recebimento para o cliente, ou envia mensagem de erro caso o voto seja inválido ou duplicado. Sempre que um voto é registrado, o servidor recalcula os resultados parciais e envia atualizações para todos os clientes conectados, aproveitando a confiabilidade do TCP para a notificação de resultados. A votação pode ser iniciada pelo servidor e tem regras bem definidas, como: número máximo de opções a selecionar, tempo limite para envio de votos e restrição a votos duplicados.

[Equipe 08] Implementar um sistema distribuído, no modelo cliente-servidor, que permita upload, download e listagem de arquivos do tipo texto utilizando socket TCP. O servidor deve escutar em uma porta fixa e aceitar múltiplas conexões. O servidor deve oferecer uma interface simples de

comandos, como `UPLOAD <nome_do_arquivo>` e `DOWNLOAD <nome_do_arquivo>`. Após o comando, o cliente envia ou solicita o arquivo. O comando `LIST` deve permitir ao servidor listar o cliente que enviou o comando todos os arquivos presentes e informações como tamanho do arquivo em bytes e outros metadados. O uso do TCP é obrigatório para garantir a transferência confiável dos dados de texto.

[Equipe 09] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que gerencie um leilão online em tempo real para um item específico. O servidor deve manter conexões TCP persistentes com todos os licitantes e gerenciar o estado atual do leilão, incluindo o lance atual (valor e ID do proponente) e o tempo restante. O cliente se conecta ao servidor para participar, podendo enviar uma oferta através do comando `BID <valor>`. O servidor deve validar se o `<valor>` é maior que o lance atual e, se válido, registrar o novo lance. O cliente também pode solicitar o estado atual do leilão usando o comando `STATUS`. É essencial implementar mecanismos de sincronização no servidor para garantir que, ao receber lances concorrentes, o registro do lance vencedor seja feito de forma atômica. A principal funcionalidade é que, sempre que um novo lance válido for registrado, o servidor deve notificar imediatamente (usando a conexão TCP persistente) todos os outros clientes conectados sobre o novo lance máximo.

[Equipe 10] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que gerencie um dicionário de termos (poucos exemplos, e.g., 5) e definições compartilhado em memória. O servidor deve aceitar múltiplas conexões e permitir que clientes consultem, insiram e atualizem definições. O TCP é usado para garantir a entrega ordenada dos comandos e a integridade das modificações. O cliente pode consultar a definição de um termo usando o comando `LOOKUP <termo>`. Para modificar o dicionário, o cliente pode inserir um novo termo (`INSERT <termo> <definição>`), ou modificar o dicionário, o cliente falhando se o termo já existir, ou modificar a definição de um termo existente usando o comando `UPDATE <termo> <nova_definição>`. É fundamental implementar mecanismos de bloqueio (locks ou mutexes) no servidor para garantir que a modificação de um termo por um cliente não seja interrompida ou sobreposta por outro cliente que tente modificar o mesmo termo simultaneamente, assegurando a integridade transacional dos dados.

[Equipe 11] Implemente um servidor de chat multiusuário usando Sockets TCP. O sistema deve permitir que vários clientes se conectem e troquem mensagens em uma única sala de chat. O servidor deve manter uma conexão TCP persistente com cada cliente para retransmitir mensagens em tempo real. O cliente se conecta e define seu nome usando o comando `CONNECT <nome_usuario>`. Em seguida, envia mensagens para o chat usando o comando `SEND <mensagem>`. A principal funcionalidade é que, ao receber uma mensagem, o servidor deve retransmitir (broadcast) essa mensagem imediatamente para todos os outros clientes conectados. Além disso, o servidor deve manter um buffer em memória das últimas N mensagens enviadas. Quando um novo cliente se conecta, o servidor deve usar a confiabilidade do TCP para enviar o histórico completo dessas últimas N mensagens somente para ele, antes de começar a receber as novas mensagens do broadcast. O cliente deve usar o comando `DISCONNECT` para encerrar a sessão.

[Equipe 12] Implemente um sistema distribuído cliente-servidor usando Sockets TCP que atue como um provedor de cotações de moedas (câmbio). O servidor deve armazenar as taxas de câmbio (simuladas) em memória e ser capaz de atender a múltiplas requisições concorrentes de forma rápida e confiável, utilizando a garantia de entrega do TCP para dados financeiros. O cliente pode solicitar uma lista de todas as moedas e suas cotações em relação a uma base usando o comando LIST, ou solicitar a taxa de câmbio atual entre duas moedas específicas (RATE <moeda_origem> <moeda_destino>). O cliente também pode solicitar a conversão de um valor de uma moeda para outra usando o comando CONVERT <origem> <destino> <valor>. O servidor deve simular a atualização periódica das cotações em memória. Dada a natureza de dados financeiros, o uso do TCP é mandatório para garantir que a resposta da cotação seja entregue ao cliente sem perdas ou corrupção, e o servidor deve ser eficiente ao atender a muitas requisições de forma concorrente.